TECHNICAL SESSION

Container Technologies on Enterprise Linux

Linux fundamentals, Docker, SELinux and Podman



ABOUT ME

Drubajyoti Debnath

Jr. Resource Person, NIELIT Agartala

- Web application development and server management
- UX engineer since 2019, freelance and working with product teams
- I build, ship and run apps like AMS in containers



CONTENTS

What we'll cover

01 Linux brush-up 0:03

02 Enterprise Linux and remote access 0:15

03 How we deployed before Docker 0:27

04 Enter Docker 0:35

05 Live demo: PHP, Apache and MySQL 0:48

06 Docker in depth 1:00

07 Debugging containers 1:16

08 Oracle Linux 9: SELinux and Podman 1:24

01

Linux brush-up

The commands you'll use every day around containers



>_ LINUX BRUSH-UP

Why start with Linux?

- Containers are just Linux processes
- Most servers are headless Linux
- Debugging a container means debugging Linux



>_ LINUX BRUSH-UP

Who am I running on?

uname prints kernel and system information

```
$ uname -a
Linux ol9 5.15.0-uek x86_64 GNU/Linux
$ uname -r              # kernel release only
$ uname -m              # architecture
```

>_ LINUX BRUSH-UP

fastfetch

A good-looking system summary in one command

```
$ sudo dnf install -y oracle-epel-release-el9
$ sudo dnf install -y fastfetch
$ fastfetch
```

The modern successor to neofetch: OS, kernel, CPU, memory and shell at a glance

>_ LINUX BRUSH-UP

More about the system

```
$ cat /etc/os-release    # distro and version
$ hostnamectl            # hostname, OS, kernel
$ whoami; id             # user, UID, groups
$ uptime                 # running time, load
```

>_ LINUX BRUSH-UP

What is a process?

A running program with its own ID: the PID

- Every process has a PID and a parent PID
- It runs as a user, with that user's permissions
- PID 1 is the first process: systemd



>_ LINUX BRUSH-UP

Listing processes with ps

```
$ ps aux                    # every process
$ ps aux | grep httpd       # find one
$ ps -ef --forest           # parent and child tree
$ pgrep -a mysqld           # PIDs by name
```

>_ LINUX BRUSH-UP

top: a live task manager



```
$ top
# P   sort by CPU        M   sort by memory
# k   kill a PID         q   quit
$ htop          # friendlier, colorful
```

>_ LINUX BRUSH-UP

Stopping a process: signals

- **SIGTERM (15)** asks nicely, so the app can clean up
- **SIGKILL (9)** stops it instantly, no cleanup
- **SIGHUP (1)** tells many daemons to reload config

>_ LINUX BRUSH-UP

Using kill

```
$ kill 4242              # SIGTERM
$ kill -9 4242           # SIGKILL
$ kill -HUP 4242         # reload
$ pkill -f php-fpm       # by name
```

Try SIGTERM first. docker stop works exactly the same way.

>_ LINUX BRUSH-UP

Memory and disk

```
$ free -h          # memory usage
$ df -h            # disk space per filesystem
$ du -sh *         # size of each folder here
$ lsblk            # disks and partitions
```

>_ LINUX BRUSH-UP

Network checks

```
$ ip a                      # interfaces and IPs
$ ss -tulpn                 # who listens on which port
$ ss -tulpn | grep 3306
$ curl -I localhost         # quick HTTP check
```

>_ LINUX BRUSH-UP

Services and logs

```
$ systemctl status httpd
$ sudo systemctl enable --now sshd
$ journalctl -u httpd -f      # follow logs
$ tail -f /var/log/messages
```

>_ LINUX BRUSH-UP

Permissions

read 4, write 2, execute 1, for owner, group and others

```
$ chmod 644 index.php    # rw-r--r--
$ chmod +x deploy.sh
$ sudo chown -R apache:apache /var/www
```

02

Enterprise Linux and remote access



RHEL, Oracle Linux, TTY, SSH and RDP

ENTERPRISE LINUX AND REMOTE ACCESS

Red Hat Enterprise Linux

- Built for servers, with a 10-year lifecycle
- Subscription for updates and support
- Free developer subscription for learning



ENTERPRISE LINUX AND REMOTE ACCESS

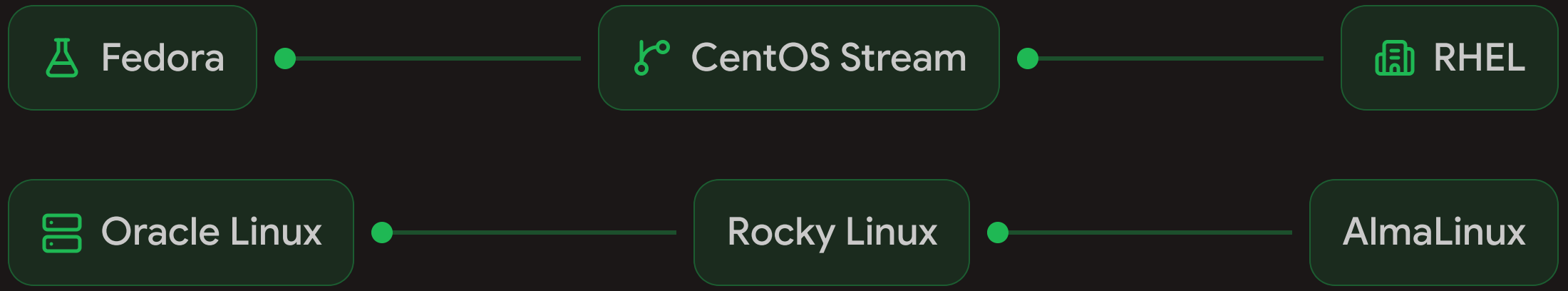
Oracle Linux

- Binary compatible with RHEL
- Free to download, use and update
- Choose a kernel: RHCK or Oracle's UEK



ENTERPRISE LINUX AND REMOTE ACCESS

One family of distros



Same tools everywhere: dnf, rpm, systemctl, firewalld and SELinux

ENTERPRISE LINUX AND REMOTE ACCESS

Installing, step 1: media

- Download the Oracle Linux 9 ISO
- VM: VirtualBox, VMware or KVM with 2 CPUs, 4 GB RAM, 30 GB disk
- Bare metal: flash a USB with Rufus, Etcher or Ventoy

ENTERPRISE LINUX AND REMOTE ACCESS

Step 2: the Anaconda installer

- Pick language and keyboard
- Installation destination: automatic partitioning
- Software selection: Server or Minimal Install
- Network and hostname: switch it on
- Root password plus an admin user

ENTERPRISE LINUX AND REMOTE ACCESS

Step 3: first boot

```
$ sudo dnf update -y
$ sudo dnf install -y oracle-epel-release-el9
$ sudo dnf install -y htop fastfetch
$ fastfetch
```

ENTERPRISE LINUX AND REMOTE ACCESS

TTY: the text console

- TeleTYpewriter: a text terminal provided by the kernel
- Ctrl Alt F1 to F6 switch consoles
- Servers boot to text mode, no GUI needed

ENTERPRISE LINUX AND REMOTE ACCESS

TTY vs PTY

```
$ tty
/dev/pts/0                # remote session: pseudo terminal
$ who                     # who is on which tty or pts
$ systemctl get-default
multi-user.target         # text mode, no GUI
```

Remember this: docker run -it asks for a pseudo-TTY

ENTERPRISE LINUX AND REMOTE ACCESS

SSH: secure shell

An encrypted remote terminal on port 22



ENTERPRISE LINUX AND REMOTE ACCESS

Connecting with SSH

```
$ sudo systemctl enable --now sshd
$ ssh admin@192.168.1.50
$ scp app.zip admin@192.168.1.50:/tmp/
```

ENTERPRISE LINUX AND REMOTE ACCESS

SSH keys: no more passwords

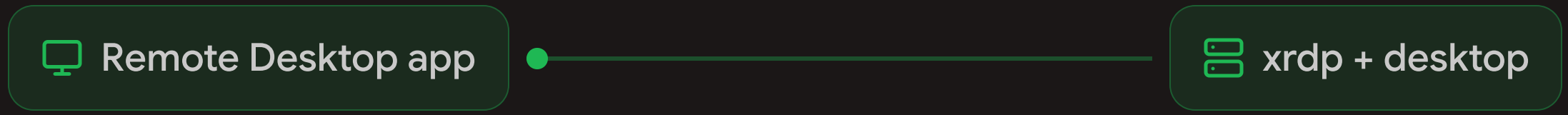


```
$ ssh-keygen -t ed25519
$ ssh-copy-id admin@192.168.1.50
$ ssh admin@192.168.1.50  # key login
```

ENTERPRISE LINUX AND REMOTE ACCESS

RDP: a full desktop

Port 3389, only when you really need a GUI



ENTERPRISE LINUX AND REMOTE ACCESS

Setting up xrdp

```
$ sudo dnf groupinstall -y "Server with GUI"
$ sudo dnf install -y xrdp
$ sudo systemctl enable --now xrdp
$ sudo firewall-cmd --permanent --add-port=3389/tcp
$ sudo firewall-cmd --reload
```

CHECKPOINT 1 / QUESTION 1

Which command shows a live view of running processes?

A `ls`

B `top`

C `cd`

D `mkdir`

CHECKPOINT 1 / ANSWER 1

Which command shows a live view of running processes?

A `ls`

B `top`

C `cd`

D `mkdir`

top refreshes every few seconds. Press q to quit.

CHECKPOINT 1 / QUESTION 2

Which port does SSH use by default?

A 80

B 443

C 22

D 3306

CHECKPOINT 1 / ANSWER 2

Which port does SSH use by default?

A 80

B 443

C 22

D 3306

SSH gives you an encrypted remote terminal on port 22.

CHECKPOINT 1 / QUESTION 3

Oracle Linux is built from which distribution?

A Ubuntu

B Windows Server

C Debian

D Red Hat Enterprise Linux

CHECKPOINT 1 / ANSWER 3

Oracle Linux is built from which distribution?

A Ubuntu

B Windows Server

C Debian

D Red Hat Enterprise Linux

Oracle Linux is built from RHEL sources, so the same commands work on both.

03

How we deployed before Docker

A PHP project, XAMPP and a lot of hope



HOW WE DEPLOYED BEFORE DOCKER

The PHP project

- Install XAMPP, or LAMPP on Linux
- Copy the project into htdocs
- Import the SQL dump in phpMyAdmin
- Tweak php.ini and httpd.conf by hand
- It works!

HOW WE DEPLOYED BEFORE DOCKER

"It works on my machine"

Then we tried production.



HOW WE DEPLOYED BEFORE DOCKER

Meanwhile, on the server

```
$ sudo dnf install httpd php php-mysqlnd mariadb-server
# dev laptop: PHP 8.2, server repo: PHP 8.0
# missing extensions: gd, zip
# different paths, users and permissions
```

HOW WE DEPLOYED BEFORE DOCKER

Problem 1: version drift

Every machine has slightly different PHP, MySQL and Apache versions. Code that runs on one breaks on another.



HOW WE DEPLOYED BEFORE DOCKER

Problem 2: tied to one machine

The environment lives inside that OS: paths, config files, installed packages. Rebuilding it takes a day.



HOW WE DEPLOYED BEFORE DOCKER

Problem 3: conflicts

Project A needs PHP 7.4, project B needs 8.2.
Both want ports 80 and 3306.



HOW WE DEPLOYED BEFORE DOCKER

Problem 4: tribal knowledge

Setup steps live in someone's head. "Oh, you also need to enable gd."



HOW WE DEPLOYED BEFORE DOCKER

Problem 5: people and scale

- New developers spend days matching versions
- Upgrades happen in place, with no clean undo
- Scaling means configuring every server by hand

04

Enter Docker

Ship the environment, not just the code



ENTER DOCKER

Docker in one sentence

Package your app with everything it needs, then run it the same way anywhere.



ENTER DOCKER

Image vs container

Image

A read-only template.

The recipe.

Container

A running instance of an image.

The dish.

One image, many containers

ENTER DOCKER

Virtual machine vs container

App	App + libraries
Guest OS + kernel	Container runtime
Hypervisor	Shared host kernel
Hardware	Hardware

No guest OS: megabytes instead of gigabytes, seconds instead of minutes

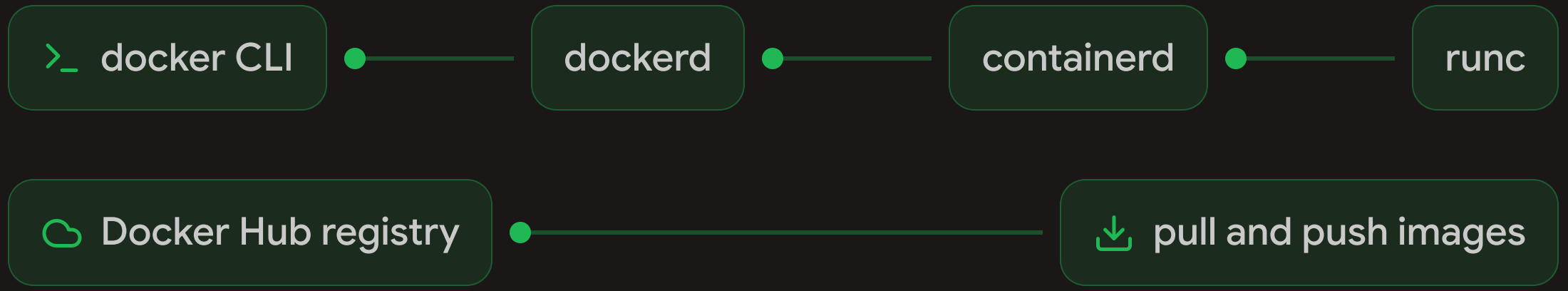
ENTER DOCKER

Under the hood

- **Namespaces**: each container sees its own processes, network and files
- **cgroups**: limit CPU and memory
- **Layered filesystem**: images stack read-only layers

ENTER DOCKER

How Docker is wired



dockerd is a daemon that runs as root. Keep that in mind for Podman later.

ENTER DOCKER

Each problem, solved

Version drift	→	Pin tags like php:8.2-apache
Tied to one machine	→	Same image on laptop, CI and server
Conflicts	→	Isolated containers, different host ports
Tribal knowledge	→	The Dockerfile is the documentation
Onboarding	→	git clone, then docker compose up
Rollback	→	Run the previous image tag

ENTER DOCKER

Dev containers

Your editor runs inside a container with the exact toolchain. Commit it, and the whole team gets identical versions.

- VS Code Dev Containers extension
- GitHub Codespaces



ENTER DOCKER

devcontainer.json

.devcontainer/devcontainer.json

```
{
  "name": "php-app",
  "image": "mcr.microsoft.com/devcontainers/php:8.2",
  "forwardPorts": [8080],
  "postCreateCommand": "composer install"
}
```

No more hunting for the exact PHP or Node version

CHECKPOINT 2 / QUESTION 1

Which common problem does Docker solve?

A "It works on my machine"

B Slow internet

C Low screen resolution

D Printer setup

CHECKPOINT 2 / ANSWER 1

Which common problem does Docker solve?

- **A** "It works on my machine"
- **B** Slow internet
- **C** Low screen resolution
- **D** Printer setup

The app ships with everything it needs, so it runs the same everywhere.

CHECKPOINT 2 / QUESTION 2

What is a running copy of an image called?

A A volume

B A container

C A network

D A Dockerfile

CHECKPOINT 2 / ANSWER 2

What is a running copy of an image called?

A A volume

B A container

C A network

D A Dockerfile

The image is the template. A container is a running copy of it.

CHECKPOINT 2 / QUESTION 3

Compared with a virtual machine, a container is...

A Heavier and slower

B Running its own full OS

C Only for Windows

D Lighter and faster to start

CHECKPOINT 2 / ANSWER 3

Compared with a virtual machine, a container is...

A Heavier and slower

B Running its own full OS

C Only for Windows

D Lighter and faster to start

Containers share the host's kernel instead of booting a whole operating system.

05

Live demo

The same XAMPP setup, rebuilt with containers



LIVE DEMO: PHP, APACHE AND MYSQL

The plan



LIVE DEMO: PHP, APACHE AND MYSQL

Step 1: network and MySQL

```
$ docker network create phpnet
$ docker run -d --name db --network phpnet \
    -e MYSQL_ROOT_PASSWORD=root \
    -e MYSQL_DATABASE=appdb \
    -v dbdata:/var/lib/mysql mysql:8.0
$ docker logs -f db     # wait for: ready for connections
```

LIVE DEMO: PHP, APACHE AND MYSQL

Step 2: the PHP page

src/index.php

```
<?php
$c = new mysqli("db", "root", "root", "appdb");
if ($c->connect_error) die($c->connect_error);
echo "MySQL " . $c->server_info . " / PHP " . PHP_VERSION;
```

The host name db works because both containers share phpnet

LIVE DEMO: PHP, APACHE AND MYSQL

Step 3: the Dockerfile

Dockerfile

```
FROM php:8.2-apache
RUN docker-php-ext-install mysqli pdo_mysql
COPY src/ /var/www/html/
```

Three lines replace a whole XAMPP install

LIVE DEMO: PHP, APACHE AND MYSQL

Step 4: build and run

```
$ docker build -t myphp:1.0 .
$ docker run -d --name web --network phpnet \
    -p 8080:80 myphp:1.0
$ curl http://localhost:8080
MySQL 8.0 / PHP 8.2
```

LIVE DEMO: PHP, APACHE AND MYSQL

Step 5: all of it in compose.yaml

compose.yaml

```yaml
services:
  web:
    build: .
    ports: ["8080:80"]
    volumes: ["./src:/var/www/html"]
    depends_on: [db]
  db:
    image: mysql:8.0
    environment: { MYSQL_ROOT_PASSWORD: root, MYSQL_DATABASE: appdb }
    volumes: ["dbdata:/var/lib/mysql"]
  phpmyadmin:
    image: phpmyadmin
    ports: ["8081:80"]
    environment: { PMA_HOST: db }
volumes:
  dbdata:
```

LIVE DEMO: PHP, APACHE AND MYSQL

One command to rule them all

```
$ docker compose up -d --build
$ docker compose ps
$ docker compose logs -f web
$ docker compose down    # add -v to drop the data
```

The whole XAMPP stack in one reproducible file

06

Docker in depth

Commands, images, containers and volumes



DOCKER IN DEPTH

Image commands

```
$ docker pull nginx:1.27
$ docker images
$ docker build -t app:1.0 .
$ docker tag app:1.0 me/app:1.0
$ docker push me/app:1.0     # after docker login
```

DOCKER IN DEPTH

More image commands

```
$ docker history app:1.0      # layers and sizes
$ docker rmi app:1.0
$ docker image prune -a       # unused images
$ docker save app:1.0 -o app.tar
$ docker load -i app.tar      # offline transfer
```

DOCKER IN DEPTH

docker run: the essentials

```
$ docker run -d --name web nginx        # background, named
$ docker run -p 8080:80 nginx           # host:container
$ docker run -e APP_ENV=dev nginx       # environment
$ docker run -it --rm ubuntu bash       # throwaway shell
```

DOCKER IN DEPTH

docker run: production flags

```
$ docker run --restart unless-stopped ...
$ docker run -m 512m --cpus 1 ...       # cgroup limits
$ docker run --network phpnet ...
$ docker run -v data:/data ...          # volume
```

DOCKER IN DEPTH

Managing containers

```
$ docker ps                 # running
$ docker ps -a              # including stopped
$ docker stop web           # SIGTERM, then SIGKILL after 10s
$ docker start web
$ docker rm -f web
$ docker cp web:/etc/nginx/nginx.conf .
$ docker system df          # disk usage
```

DOCKER IN DEPTH

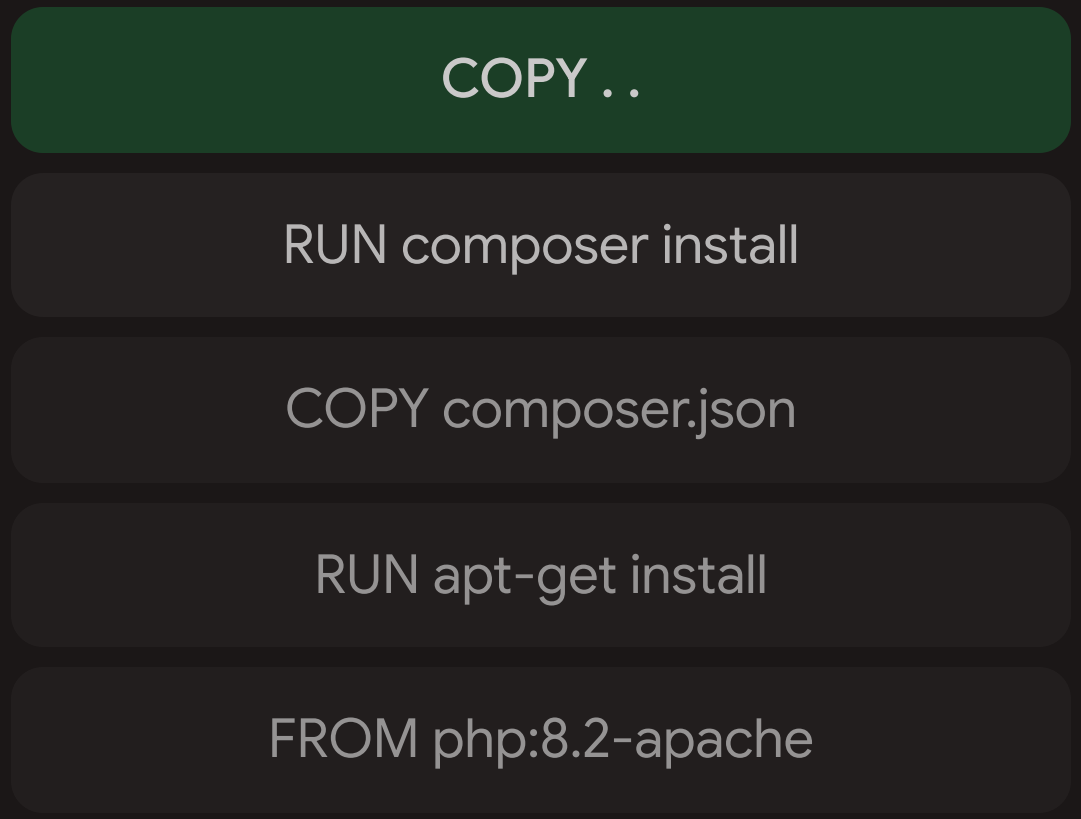
Dockerfile instructions

Dockerfile

```
FROM php:8.2-apache             # base image
WORKDIR /var/www/html           # working folder
RUN docker-php-ext-install zip # build-time command
COPY . .                        # files from host
ENV APP_ENV=prod                # environment variable
EXPOSE 80                       # documents the port
CMD ["apache2-foreground"]      # default start command
```

DOCKER IN DEPTH

Layers and the cache



Each instruction is a cached layer.
Change one, and only it and the layers above it rebuild.

DOCKER IN DEPTH

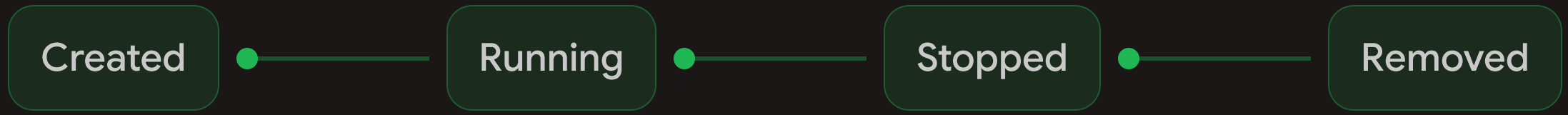
Building images well

- Copy dependency files before source code
- Chain RUN commands and clean caches
- Use .dockerignore for .git, node_modules, .env
- Pin versions, avoid latest in production
- One main process per container

DOCKER IN DEPTH

Container lifecycle

docker run is create plus start



DOCKER IN DEPTH

Inside a container

- A thin writable layer on top of the image
- docker rm deletes that layer and its data
- The main process is PID 1. When it exits, the container stops.

DOCKER IN DEPTH

Volumes: data that survives

Containers are disposable. Your database is not.



DOCKER IN DEPTH

Named volumes

Managed by Docker. Best for databases.

```
$ docker volume create dbdata
$ docker run -v dbdata:/var/lib/mysql mysql:8.0
$ docker volume ls
$ docker volume inspect dbdata
```

DOCKER IN DEPTH

Bind mounts

A host folder mapped inside. Best for live code.

```
$ docker run -v $(pwd)/src:/var/www/html \
    -p 8080:80 php:8.2-apache
# edit on the host, refresh the browser
```

DOCKER IN DEPTH

tmpfs and cleanup

```
$ docker run --tmpfs /tmp ...       # memory only
$ docker volume rm dbdata
$ docker volume prune               # unused volumes
```

DOCKER IN DEPTH

Prove it

```
$ docker rm -f db
$ docker run -d --name db -v dbdata:/var/lib/mysql \
    -e MYSQL_ROOT_PASSWORD=root mysql:8.0
# the tables are still there
```

CHECKPOINT 3 / QUESTION 1

Which file has the steps to build an image?

A `index.php`

B `README.md`

C `Dockerfile`

D `.bashrc`

CHECKPOINT 3 / ANSWER 1

Which file has the steps to build an image?

A `index.php`

B `README.md`

C `Dockerfile`

D `.bashrc`

Each line in a Dockerfile is a step, like FROM, COPY or RUN.

CHECKPOINT 3 / QUESTION 2

Which command starts everything in compose.yaml?

A `docker compose up`

B `docker stop`

C `docker rm`

D `docker logs`

CHECKPOINT 3 / ANSWER 2

Which command starts everything in compose.yaml?

A `docker compose up`

B `docker stop`

C `docker rm`

D `docker logs`

Add -d to keep it running in the background.

CHECKPOINT 3 / QUESTION 3

Where should database data live so it isn't lost?

A Inside the container

B In a volume

C Inside the image

D In `/tmp`

CHECKPOINT 3 / ANSWER 3

Where should database data live so it isn't lost?

A Inside the container

B In a volume

C Inside the image

D In `/tmp`

A volume keeps the data even when the container is removed.

07

Debugging containers

Logs, shells, inspection and common failures



DEBUGGING CONTAINERS

Start with the logs

Whatever the app prints to stdout and stderr

```
$ docker logs web
$ docker logs -f --tail 100 web
$ docker logs --since 10m web
$ docker compose logs -f
```

DEBUGGING CONTAINERS

Get inside with exec

```
$ docker exec -it web bash          # or sh on alpine
$ docker exec web cat /etc/hosts
$ docker exec -it db mysql -uroot -p
```

DEBUGGING CONTAINERS

Look closer with inspect

```
$ docker inspect web
$ docker inspect -f '{{.State.ExitCode}}' web
$ docker inspect -f '{{.Mounts}}' web
$ docker port web
```

DEBUGGING CONTAINERS

Watch it live

```
$ docker stats          # CPU and memory per container
$ docker top web        # processes inside
$ docker events         # what the daemon is doing
$ docker diff web       # files changed vs the image
```

DEBUGGING CONTAINERS

When it keeps crashing

- **Exit 137**: killed, often out of memory
- **Exit 1**: application error, read the logs
- **Exit 0**: the main process finished on its own

```
$ docker run -it --entrypoint sh myphp:1.0   # bypass CMD
```

DEBUGGING CONTAINERS

Common failures

Port allocated	→	ss -tulpn, then pick another host port
Can't reach DB	→	Same network? Use the service name
Mount denied	→	UID mismatch or SELinux
Changes missing	→	Rebuild with --build or bind mount
Disk full	→	docker system df, then prune

DEBUGGING CONTAINERS

Wait until the DB is ready

compose.yaml

```
db:
  image: mysql:8.0
  healthcheck:
    test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
    interval: 5s
    retries: 10
web:
  depends_on:
    db: { condition: service_healthy }
```

08

SELinux and Podman



How containers behave on Oracle Linux 9

ORACLE LINUX 9: SELINUX AND PODMAN

What is SELinux?

Mandatory access control. Every file and process has a label, and policy decides who can touch what, even root.



ORACLE LINUX 9: SELINUX AND PODMAN

chmod 777 and still denied?

Classic permissions say yes. SELinux says no.
Both must allow the access.



ORACLE LINUX 9: SELINUX AND PODMAN

SELinux modes

```
$ getenforce
Enforcing
$ sestatus
$ sudo setenforce 0      # Permissive, testing only
```

Fix the label. Never disable SELinux in production.

ORACLE LINUX 9: SELINUX AND PODMAN

Labels, also called contexts

user : role : type : level. The type matters most.

```
$ ls -Z /var/www/html
system_u:object_r:httpd_sys_content_t:s0 index.php
$ ps -eZ | grep httpd
system_u:system_r:httpd_t:s0  1234  httpd
```

ORACLE LINUX 9: SELINUX AND PODMAN

Finding denials



```
$ sudo ausearch -m avc -ts recent
$ sudo dnf install -y setroubleshoot-server
$ sudo sealert -a /var/log/audit/audit.log
```

ORACLE LINUX 9: SELINUX AND PODMAN

Fixing labels

```
$ sudo restorecon -Rv /var/www/html
$ sudo semanage fcontext -a -t httpd_sys_rw_content_t \
    "/var/www/html/uploads(/.*)?"
$ sudo restorecon -Rv /var/www/html/uploads
$ sudo setsebool -P httpd_can_network_connect_db on
```

ORACLE LINUX 9: SELINUX AND PODMAN

SELinux and container volumes

```
$ podman run -v ./src:/var/www/html php:8.2-apache
# Permission denied: files are labelled user_home_t
$ podman run -v ./src:/var/www/html:Z ...   # private
$ podman run -v ./shared:/data:z ...        # shared
```

⚠ Never use :Z on system folders like /home or /etc

ORACLE LINUX 9: SELINUX AND PODMAN

Meet Podman

Oracle Linux 9 ships Podman, not Docker.
Same images, same commands, no daemon.



ORACLE LINUX 9: SELINUX AND PODMAN

Installing Podman

```
$ sudo dnf install -y container-tools  # podman, buildah, skopeo
$ podman run -d -p 8080:80 docker.io/library/nginx
$ sudo dnf install -y podman-docker    # docker calls podman
```

ORACLE LINUX 9: SELINUX AND PODMAN

Docker vs Podman

Docker

Central daemon, dockerd

Traditionally runs as root

Compose built in

Docker Desktop and Swarm

Podman

Daemonless: fork and exec

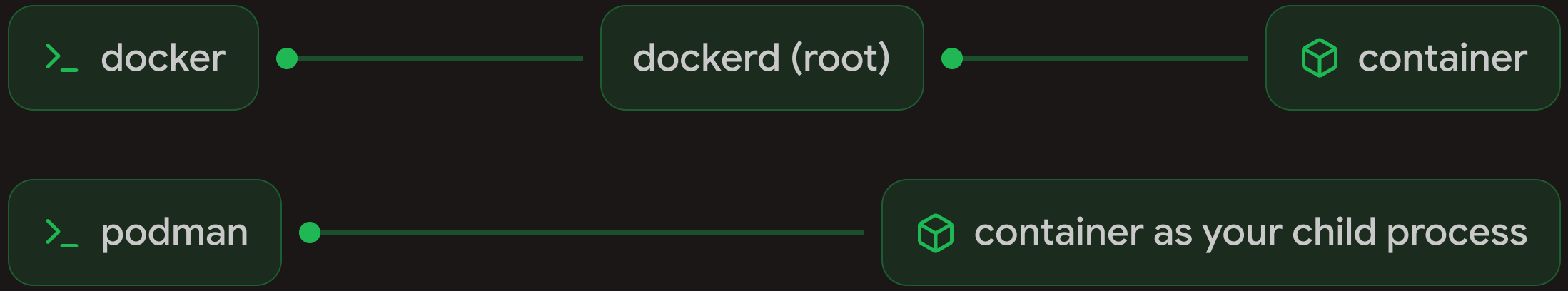
Rootless by default

Kubernetes-style pods

systemd with Quadlet

ORACLE LINUX 9: SELINUX AND PODMAN

Daemon vs daemonless



No single root process to crash or to attack

ORACLE LINUX 9: SELINUX AND PODMAN

Pods, Kubernetes and compose

```
$ podman pod create --name app -p 8080:80
$ podman run -d --pod app mysql:8.0
$ podman kube generate app > app.yaml
$ podman compose up -d
```

Quadlet files in /etc/containers/systemd run containers as services

CHECKPOINT 4 / QUESTION 1

A container keeps crashing. What do you check first?

A Restart the computer

B Reinstall Docker

C Delete the image

D `docker logs`

CHECKPOINT 4 / ANSWER 1

A container keeps crashing. What do you check first?

A Restart the computer

B Reinstall Docker

C Delete the image

D `docker logs`

The logs usually show the error straight away.

CHECKPOINT 4 / QUESTION 2

Which container tool comes with Oracle Linux 9?

A Podman

B VirtualBox

C XAMPP

D Notepad

CHECKPOINT 4 / ANSWER 2

Which container tool comes with Oracle Linux 9?

- **A** Podman
- B VirtualBox
- C XAMPP
- D Notepad

Podman runs the same images with the same commands, without a daemon.

CHECKPOINT 4 / QUESTION 3

On Oracle Linux, what can block access even after chmod 777?

A The keyboard layout

B The screen saver

C SELinux

D The time zone

CHECKPOINT 4 / ANSWER 3

On Oracle Linux, what can block access even after chmod 777?

A The keyboard layout

B The screen saver

C SELinux

D The time zone

Mount volumes with :Z or :z so SELinux lets the container use them.

WRAP-UP

Key takeaways

- Linux basics are the foundation of containers
- XAMPP-style setups tie apps to one machine
- Images package, containers run, volumes persist
- Debug with logs, exec, inspect and stats
- On Oracle Linux: Podman plus SELinux labels

THANK YOU

Questions and Discussion

Container Technologies on Enterprise Linux



Next session

Deploying a Live Project with Docker